



UNIVERSITATEA TEHNICA
DIN CLUJ-NAPOCA
CENTRUL UNIVERSITAR NORD DIN BAIA MARE
FACULTATEA DE INGINERIE

DEPARTAMENTUL CALCULATOARE

Site de piese auto AUTOKAT

Autor: **Cătălin-Beniamin Danciu**

2026

Cuprins

Capitolul 1	Introducere	1
1.1	Context general	1
1.2	Scopul și motivația proiectului	1
1.3	Obiective	1
1.4	Lista MoSCoW	2
1.5	Cazuri de utilizare	2
Capitolul 2	Arhitectura aplicației	4
2.1	Prezentare generală	4
2.2	Structura generală a aplicației	6
2.3	Frontend	6
2.4	Backend	6
2.5	Securitate	6
2.6	Baza de date	7
2.7	Avantajele arhitecturii	7
Capitolul 3	Implementare Frontend	8
3.1	Tehnologii utilizate în Frontend	8
3.2	Structura paginilor	8
3.3	Componentele principale ale interfeței	8
3.3.1	Header și navigație	8
3.3.2	Catalogul de produse	9
3.3.3	Pagina de detalii produs	9
3.3.4	Coșul de cumpărături	9
3.4	Utilizarea aplicației – utilizator Guest	9
3.5	Utilizarea aplicației – utilizator autentificat	9
3.6	Utilizarea aplicației – Administrator	9
3.7	Concluzii privind frontend-ul	10
Capitolul 4	Implementare Backend	11
4.1	Tehnologii utilizate în Backend	11
4.2	Spring Boot	11
4.3	Arhitectura backend-ului	11
4.4	Controller Layer	11
4.5	Service Layer	12
4.6	Repository Layer și JPA	12
4.7	Model Layer	12
4.8	Securitate și autentificare	12
4.9	Gestionarea sesiunii	13
4.10	Concluzii privind backend-ul	13
Capitolul 5	Concluzii	14
5.1	Obiective atinse	14
5.2	Rezultatele proiectului	14
5.3	Tehnologii utilizate	14

Capitolul 1. Introducere

1.1. Context general

Aplicația web *Shop Online Produse Auto* a fost realizată pentru a oferi utilizatorilor o modalitate modernă, rapidă și eficientă de a achiziționa produse auto, precum consumabile, accesorii și piese de întreținere. În contextul dezvoltării accelerate a comerțului electronic, tot mai mulți utilizatori preferă platformele online pentru achiziționarea produselor, datorită accesibilității și ușurinței în utilizare.

Domeniul pieselor auto implică gestionarea unui număr mare de produse, diferențiate prin categorii, branduri, prețuri și stocuri disponibile. Din acest motiv, este necesară o aplicație care să permită organizarea eficientă a informațiilor și să ofere un flux de utilizare intuitiv pentru clienți. Aplicația AutoKat răspunde acestei nevoi printr-un catalog structurat de produse și un sistem de coș de cumpărături ușor de utilizat.

Aplicația oferă funcționalități atât pentru utilizatorii neautentificați sau autentificați, cât și pentru administratorii platformei. Utilizatorii pot vizualiza produsele disponibile, pot consulta detaliile acestora și pot adăuga articolele dorite în coș, în timp ce administratorul are la dispoziție un panou dedicat pentru gestionarea produselor din magazin.

1.2. Scopul și motivația proiectului

Scopul principal al acestui proiect este realizarea unei aplicații web funcționale care să simuleze comportamentul unui magazin online de piese auto. Proiectul are rolul de a integra și aplica în practică noțiunile studiate în cadrul disciplinei *Programare Web*, precum dezvoltarea aplicațiilor backend, realizarea interfețelor frontend și securizarea accesului la resurse.

Motivația realizării aplicației constă în dorința de a crea o soluție completă care să rezolve problema gestionării produselor auto într-un mediu online. Prin intermediul aplicației AutoKat se urmărește oferirea unei experiențe coerente utilizatorilor, de la vizualizarea produselor până la finalizarea comenzii, respectând bunele practici de dezvoltare software.

1.3. Obiective

Această aplicație urmărește să ofere o experiență fluentă și intuitivă atât pentru utilizatorii obișnuiți (Guest sau User), cât și pentru administratori (Admin). Obiectivele principale ale aplicației sunt:

- Vizualizarea produselor auto disponibile în magazin
- Accesarea detaliilor unui produs
- Adăugarea produselor în coșul de cumpărături
- Modificarea cantităților produselor din coș, în funcție de stoc
- Ștergerea produselor din coș
- Calcularea automată a prețului total al comenzii
- Autentificarea și autorizarea utilizatorilor
- Gestionarea produselor de către administrator (operații CRUD)

- Protejarea zonelor sensibile ale aplicației

1.4. Lista MoSCoW

Pentru prioritizarea cerințelor aplicației a fost utilizată metoda MoSCoW, care împarte funcționalitățile în patru categorii, în funcție de importanța lor pentru aplicație.

Must have	Should have	Could have	Won't have
Listare produse	Căutare produse	Istoric comenzi	Plăți online
Detalii produs	Filtrare categorii	Favorite	Integrare curier
Adăugare în coș	Actualizare cantitate	Profil utilizator	AWB automat
CRUD produse	Mesaje de confirmare	Recenzii produse	Chat suport
Autentificare	Interfață responsive		

Tabela 1.1: Lista MoSCoW

1.5. Cazuri de utilizare

Aplicația este destinată atât utilizatorilor obișnuiți, cât și administratorilor platformei. Principalele cazuri de utilizare sunt:

- **Guest / User:**
 - vizualizează catalogul de produse
 - accesează detalii despre produse
 - adaugă produse în coșul de cumpărături
 - modifică sau elimină produse din coș
 - finalizează comanda
- **Admin:**
 - se autentifică în aplicație
 - adaugă produse noi
 - editează produse existente
 - șterge produse
 - gestionează stocurile

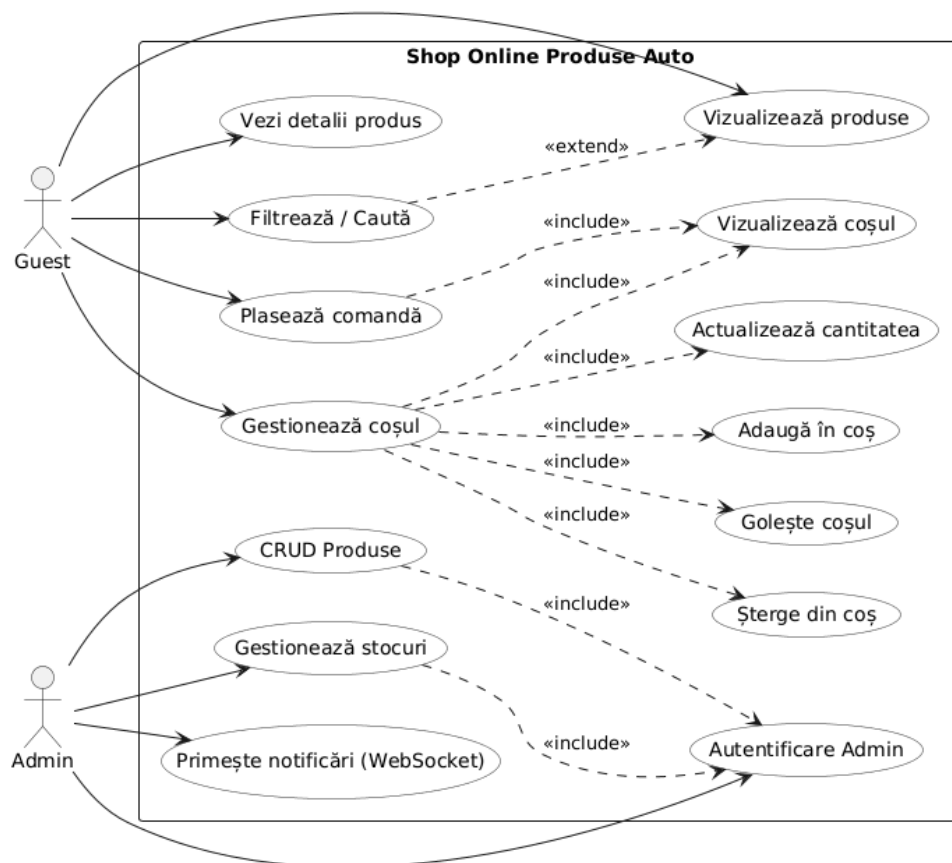


Figura 1.1: Diagrama cazurilor de utilizare

Capitolul 2. Arhitectura aplicației

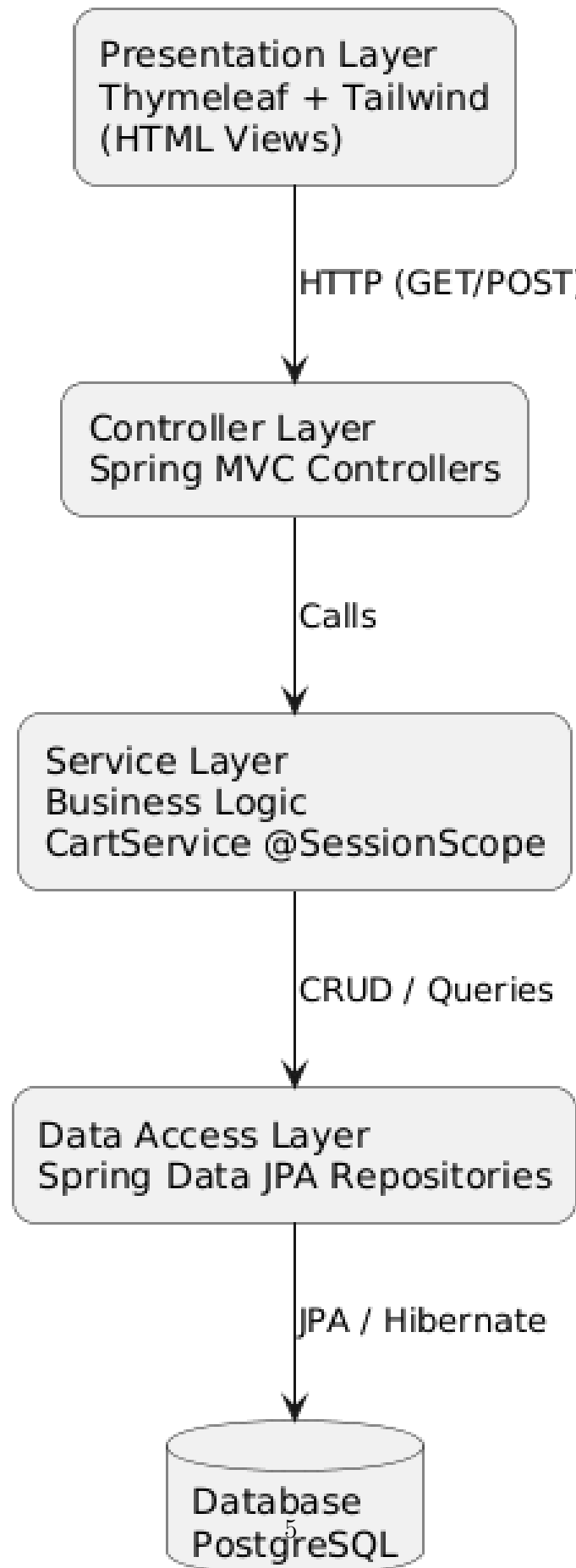
2.1. Prezentare generală

Aplicația **AutoKat** este construită pe o arhitectură de tip *client-server*, unde partea de server se ocupă de logica aplicației și de accesul la baza de date, iar partea de client este responsabilă de afișarea interfeței și interacțiunea cu utilizatorul.

Această abordare permite o organizare clară a codului, o dezvoltare mai ușoară și posibilitatea de a extinde aplicația în viitor fără modificări majore.

În figura 2.1 este prezentată arhitectura generală a aplicației.

Arhitectură pe straturi (MVC) - AutoKat



2.2. Structura generală a aplicației

Aplicația este împărțită în următoarele componente principale:

- Frontend – interfața utilizator
- Backend – logica de business și securitatea
- Baza de date – stocarea informațiilor

Fiecare componentă are un rol bine definit și comunică cu celelalte prin mecanisme standard.

2.3. Frontend

Frontend-ul aplicației este realizat folosind pagini HTML generate cu ajutorul **Thymeleaf** și stilizate cu **Tailwind CSS**.

Rolul frontend-ului este de a:

- afișa produsele disponibile
- permite utilizatorilor să vadă detalii despre produse
- gestioneze coșul de cumpărături
- ofere formulare de autentificare și înregistrare

Interfața este responsive și ușor de utilizat, fiind adaptată atât pentru desktop, cât și pentru dispozitive mobile.

2.4. Backend

Backend-ul aplicației este implementat folosind **Spring Boot** și reprezintă partea centrală a sistemului.

Responsabilitățile principale ale backend-ului sunt:

- gestionarea produselor
- logica de business pentru coșul de cumpărături
- autentificarea și autorizarea utilizatorilor
- comunicarea cu baza de date

Aplicația este structurată pe straturi clare:

- Controllers – gestionează cererile HTTP
- Services – conțin logica aplicației
- Repositories – asigură accesul la date
- Entities – modelează structura datelor

Această structură face codul ușor de înțeles și de întreținut.

2.5. Securitate

Securitatea aplicației este asigurată cu ajutorul **Spring Security**. Accesul la anumite funcționalități este restricționat în funcție de rolul utilizatorului.

- utilizatorii neautentificați pot doar vizualiza produsele
- utilizatorii autentificați pot utiliza coșul de cumpărături
- administratorii pot gestiona produsele

Astfel, aplicația protejează datele și funcționalitățile sensibile.

2.6. Baza de date

Pentru stocarea informațiilor s-a utilizat o bază de date relațională. Aceasta conține date despre:

- utilizatori
- produse
- roluri și drepturi de acces

Accesul la baza de date este realizat prin **Spring Data JPA**, ceea ce permite operații simple de tip CRUD fără a scrie interogări SQL manuale.

2.7. Avantajele arhitecturii

Arhitectura aleasă oferă mai multe avantaje:

- cod organizat și ușor de extins
- separarea clară a responsabilităților
- securitate bine definită
- posibilitatea de a adăuga funcționalități noi

Această structură face aplicația AutoKat potrivită pentru dezvoltări viitoare și utilizare practică.

Capitolul 3. Implementare Frontend

Implementarea interfeței utilizator pentru aplicația **AutoKat** a avut ca scop realizarea unei experiențe simple, clare și intuitive pentru utilizatori. Frontend-ul este integrat direct în aplicația Spring Boot și folosește pagini HTML generate dinamic cu ajutorul Thymeleaf, stilizate folosind Tailwind CSS.

Aplicația nu este de tip SPA, însă navigarea este clară și rapidă, fiind potrivită pentru un magazin online de dimensiuni medii.

3.1. Tehnologii utilizate în Frontend

Pentru dezvoltarea interfeței grafice au fost utilizate următoarele tehnologii:

- **HTML5** – structurarea conținutului paginilor
- **Thymeleaf** – motor de template pentru generarea dinamică a paginilor
- **Tailwind CSS** – framework CSS pentru stilizare modernă și responsive
- **Spring MVC** – integrarea paginilor cu backend-ul

Avantajele acestei abordări sunt:

- integrare directă cu backend-ul Spring Boot
- cod frontend ușor de întreținut
- afișare dinamică a datelor din baza de date
- suport pentru condiționarea elementelor UI în funcție de autentificare

3.2. Structura paginilor

Frontend-ul aplicației este organizat sub forma mai multor pagini HTML, fiecare având o responsabilitate clară.

Structura principală a paginilor este:

- **catalog.html** – afișarea listei de produse
- **product.html** – afișarea detaliilor unui produs
- **cart.html** – vizualizarea și gestionarea coșului de cumpărături
- **login.html** – autentificarea utilizatorilor
- **register.html** – înregistrarea utilizatorilor
- **admin/products.html** – gestionarea produselor de către administrator

Toate paginile folosesc un header comun care conține navigarea principală.

3.3. Componentele principale ale interfeței

Aplicația este construită folosind componente UI reutilizabile la nivel de pagină.

3.3.1. Header și navigație

Header-ul este prezent pe toate paginile și conține:

- link către catalog
- acces către coșul de cumpărături
- butoane de Login / Logout

- acces către panoul de administrare (doar pentru Admin)

Afișarea elementelor este condiționată prin Thymeleaf, în funcție de starea utilizatorului (autentificat sau nu).

3.3.2. Catalogul de produse

Pagina de catalog afișează produsele disponibile folosind:

- listare dinamică prin `th:each`
- afișarea informațiilor principale: nume, brand, categorie, preț, stoc
- buton de acces către pagina de detalii

Produsele inactive sau fără stoc nu sunt afișate utilizatorilor obișnuiți.

3.3.3. Pagina de detalii produs

Pagina de detalii afișează:

- informații complete despre produs
- prețul și stocul disponibil
- buton pentru adăugarea produsului în coș

Adăugarea în coș este realizată printr-un formular HTML care trimite o cerere POST către backend.

3.3.4. Coșul de cumpărături

Pagina `cart.html` permite utilizatorului:

- vizualizarea produselor adăugate
- modificarea cantității produselor
- ștergerea produselor din coș
- vizualizarea prețului total

Coșul este gestionat la nivel de sesiune, astfel încât fiecare utilizator are propriul coș independent.

3.4. Utilizarea aplicației – utilizator Guest

Utilizatorul neautentificat (Guest) are acces limitat la funcționalitățile publice.

Scenariul de utilizare pentru Guest:

- accesarea paginii de catalog
- vizualizarea produselor disponibile
- accesarea detaliilor unui produs
- redirectionare către Login la încercarea de accesare a coșului

3.5. Utilizarea aplicației – utilizator autentificat

Utilizatorul autentificat are acces la funcționalități suplimentare.

Funcționalități disponibile:

- adăugarea produselor în coș
- modificarea cantităților
- finalizarea comenzii (simulată)
- logout din aplicație

3.6. Utilizarea aplicației – Administrator

Administratorul are acces la zona de administrare a aplicației.

Funcționalitățile principale pentru Admin sunt:

- adăugarea de produse noi
- editarea produselor existente
- ștergerea produselor
- gestionarea stocului

Accesul la aceste funcționalități este restricționat prin Spring Security și afișat doar utilizatorilor cu rol ADMIN.

3.7. Concluzii privind frontend-ul

Frontend-ul aplicației AutoKat este simplu, clar și eficient, fiind adaptat cerințelor unui magazin online. Utilizarea Thymeleaf și Tailwind CSS permite dezvoltarea rapidă a interfeței și integrarea directă cu backend-ul Spring Boot, fără a introduce complexitate inutilă.

Capitolul 4. Implementare Backend

Backend-ul aplicației **AutoKat** reprezintă componenta centrală a sistemului, responsabilă de logica de business, gestionarea securității și accesul la baza de date. Implementarea este realizată folosind framework-ul Spring Boot, care permite dezvoltarea rapidă a aplicațiilor web Java, cu o structură clară și ușor de întreținut.

Backend-ul este integrat direct cu frontend-ul Thymeleaf și răspunde cererilor HTTP generate de formularele și link-urile din aplicație.

4.1. Tehnologii utilizate în Backend

Pentru dezvoltarea părții de server au fost utilizate următoarele tehnologii:

- **Java** – limbajul principal de programare
- **Spring Boot** – framework-ul principal al aplicației
- **Spring MVC** – gestionarea cererilor HTTP și a controllerelor
- **Spring Security** – autentificare și autorizare
- **Spring Data JPA** – accesul la baza de date
- **PostgreSQL** – sistem de gestiune a bazei de date
- **Maven** – managementul dependențelor

4.2. Spring Boot

Spring Boot a fost ales datorită simplității și integrării rapide a componentelor. Acesta elimină configurările complexe și permite rularea aplicației cu un minim de setări.

Avantajele utilizării Spring Boot:

- configurare automată a componentelor
- structură clară a proiectului
- pornire rapidă a aplicației
- integrare facilă cu Spring Security și JPA

Aplicația este pornită din clasa principală **SiteAutoApplication**.

4.3. Arhitectura backend-ului

Backend-ul aplicației AutoKat este structurat pe mai multe straturi:

- **Controller Layer** – gestionează cererile HTTP
- **Service Layer** – conține logica de business
- **Repository Layer** – comunică cu baza de date
- **Model Layer** – entitățile aplicației

Această separare permite o mentenanță ușoară și un cod clar.

4.4. Controller Layer

Controller-ele sunt annotate cu **@Controller** și gestionează rutele aplicației.

Principalele controllere implementate sunt:

- **CatalogController** – afișarea produselor și detaliilor acestora
- **ProductController** – vizualizarea detaliilor unui produs

- **CartController** – gestionarea coșului de cumpărături
 - **AuthController** – autentificare și înregistrare utilizatori
 - **AdminProductController** – administrarea produselor
- Controller-ele returnează pagini HTML generate cu Thymeleaf.

4.5. Service Layer

Stratul de servicii conține logica principală a aplicației.

Principalul serviciu implementat:

- **CartService**
Funcționalitățile oferite de CartService:
- adăugarea produselor în coș
- actualizarea cantităților
- eliminarea produselor din coș
- calcularea sumei totale
- verificarea stocului disponibil

Serviciul este definit cu **@Service** și are scope de sesiune, astfel încât fiecare utilizator are un coș propriu.

4.6. Repository Layer și JPA

Accesul la baza de date este realizat prin Spring Data JPA.

Repository-urile sunt interfețe care extind **JpaRepository**:

- **ProductRepository**
- **UserAccountRepository**
Avantajele utilizării JPA:
- eliminarea codului SQL manual
- operații CRUD generate automat
- interogări bazate pe numele metodelor
- mapare automată între entități și tabele

4.7. Model Layer

Modelele aplicației sunt mapate pe tabelele din baza de date folosind adnotări JPA.

Principalele entități sunt:

- **Product** – informații despre produsele auto
- **UserAccount** – datele utilizatorilor
- **CartItem** – elementele din coșul de cumpărături

Fiecare entitate utilizează adnotări precum:

- **@Entity**
- **@Id**
- **@GeneratedValue**
- **@Column**

4.8. Securitate și autentificare

Securitatea aplicației este implementată folosind Spring Security.

Funcționalități principale:

- autentificare prin formular

- roluri diferite pentru utilizatori (USER, ADMIN)
- restricționarea accesului la rutele de administrare
- logout securizat

Accesul este controlat astfel:

- utilizatorii neautentificați pot accesa doar catalogul
- utilizatorii autentificați pot accesa coșul
- administratorii pot gestiona produsele

4.9. Gestionarea sesiunii

Aplicația utilizează sesiuni HTTP pentru:

- menținerea autentificării utilizatorului
- stocarea coșului de cumpărături

Coșul de cumpărături este resetat automat la logout.

4.10. Concluzii privind backend-ul

Backend-ul aplicației AutoKat este simplu, eficient și bine structurat. Utilizarea Spring Boot, Spring Security și JPA permite dezvoltarea rapidă a unui sistem sigur și ușor de întreținut, adaptat unui magazin online de produse auto.

Capitolul 5. Concluzii

Proiectul **AutoKat** reprezintă o aplicație web complet funcțională pentru gestionarea și vânzarea produselor auto, realizată folosind tehnologii moderne din ecosistemul Java și web. Aplicația demonstrează modul în care un magazin online poate fi implementat într-un mod organizat, sigur și ușor de utilizat.

Pe parcursul dezvoltării, s-a pus accent pe simplitate, claritate și separarea corectă a responsabilităților între componentele aplicației.

5.1. Obiective atinse

Obiectivele stabilite la începutul proiectului au fost realizate cu succes:

- realizarea unui catalog de produse auto
 - afișarea detaliilor fiecărui produs
 - implementarea unui coș de cumpărături funcțional
 - calcularea automată a prețului total
 - autentificare și înregistrare utilizatori
 - separarea rolurilor USER și ADMIN
 - administrarea produselor de către administrator
-

5.2. Rezultatele proiectului

Aplicația oferă următoarele funcționalități principale:

- utilizatorii pot vizualiza produsele și detaliile acestora
- produsele pot fi adăugate în coș, respectând stocul disponibil
- coșul de cumpărături este gestionat la nivel de sesiune
- administratorii pot adăuga, edita și șterge produse
- accesul este controlat prin Spring Security

Interfața este simplă, intuitivă și adaptată pentru diferite dimensiuni de ecran, folosind Tailwind CSS.

5.3. Tehnologii utilizate

În cadrul proiectului au fost utilizate tehnologii populare și stabile:

- **Spring Boot** pentru backend
- **Spring MVC** pentru controlul cererilor
- **Spring Security** pentru autentificare și autorizare
- **Spring Data JPA** pentru accesul la baza de date
- **PostgreSQL** ca sistem de gestiune a bazei de date
- **Thymeleaf** pentru generare